```
            title=article.title,
            author=article.author.username,
            category=article.category.name,
            content=article.content)
        return HttpResponse(response)
```

The way to map this to a URL in the 'urls.py' file is a little different. Where you used 'views.article_detail' in the case of a function-based view, you will now need to use 'views.ArticleDetail.as_view()'

Depending on your proclivity toward object-oriented programming you might find this better / cleaner or worse than the function-based approach. Wait till you see how class-based views work with templates though, as that might change your mind.

With a template placed in the correct place, this class-based view reduces to:

```
from django.views.generic import DetailView

class ArticleDetail(DetailView):
    model = Article
```

This is possible in part because we named the slugs we capture from the URL. Django knows how to handle this case automatically. This also requires that the template files be placed in the correct directory, and have the correct name. In this case you would need to name the file 'article_detail.html' and place it in a folder called 'cms' (same as the name of our app) in the templates folder. Of course all of this can be changed and configured if you want, but if you follow the conventions, it can be quite rewarding.

All of this will become a lot clearer when we study templates in more detail. Before that though, we will look deeper into class-based views, they seem to have some interesting things going on.

Generic Class-Based Views

A lot of the work in rendering different web pages is actually quite repetitive. Whether it is a list of articles or a list of galleries, we need to perform a similar set of tasks. First we need to fetch a list of items, and then we need to pass them to the template to build our webpage, and then we need to return this to the user as a response that will be rendered in the browser.

What's different between and Article, and a Gallery in this case is the name of the model, and the template used to render this list of items. So to